



**Evaluación conjunta: Auditoría de Seguridad y Despliegue en Kubernetes del
Sistema InvenTrack**

Marcos Escobar

Universidad de las Fuerzas Armadas ESPE

30735 — Desarrollo de Software Seguro

Ing. Geovanny Cudco

31 de julio de 2026



Evaluación conjunta: Auditoría de Seguridad y Despliegue en Kubernetes del

Sistema InvenTrack

Índice

Introducción	5
Objetivo General	5
Objetivos Específicos	5
Enlace al Repositorio de Manifiestos de Kubernetes	6
Dockerización de Componentes	6
Backend (Node.js/Express)	6
Frontend (React/Vite)	7
Base de Datos (MySQL)	8
Evidencia: Contenedores en Ejecución	8
Manifiestos de Kubernetes	9
Namespaces	9
Deployments y Services	10
ConfigMaps y Secrets	13
PersistentVolumeClaim (PVC)	14
Ingress	15
Despliegue en el Clúster y Configuración de Ingress	16
Descripción del Clúster	16
Controlador Ingress	16
Resolución del Dominio conjunta3p.espe.edu.ec	16

Evidencia: Recursos en Kubernetes	16
Evidencia: Aplicación en el Navegador	16
Auditoría de Seguridad: Pentesting con Metodología PTES	17
Fase 1: Interacciones Preliminares (Pre-engagement) — 5 %	17
Alcance del Pentesting	17
Objetivos y Reglas de Juego	17
Activos a Probar	17
Autorización	17
Fase 2: Recopilación de Inteligencia (Intelligence Gathering) — 8 %	17
Mapeo de Puertos (Nmap)	17
Detección de Tecnologías (WhatWeb / Wappalyzer)	17
Cabeceras HTTP y Rutas de Ingress	17
Fase 3: Modelado de Amenazas (Threat Modeling) — 7 %	17
Fase 4: Análisis de Vulnerabilidades (Vulnerability Analysis) — 12 %	18
Escaneo con Nikto	18
Escaneo con OWASP ZAP / Burp Suite	18
Análisis de Cabeceras de Seguridad	18
Resumen de Vulnerabilidades Detectadas	18
Fase 5: Explotación (Exploitation) — 15 %	18
Intento de Bypass de Autenticación	18
Fuerza Bruta contra el Login (Hydra / Burp Intruder)	18
Inyección SQL (Login / Chatbot)	19
Pruebas de IDOR (Endpoints de Auditoría / Kardex)	19
Fase 6: Post-Explotación (Post-Exploitation) — 8 %	19
Análisis de Impacto	19
Validación del Hasheo con bcrpt	19
Escalación de Privilegios (admin vs almacenero)	19
Fase 7: Reporte (Reporting) — 5 %	19

Clasificación de Hallazgos por Severidad	19
Validación de Controles de Seguridad	19
Conclusiones y Recomendaciones	20
Conclusiones	20
Recomendaciones de Mejora para InvenTrack	20

Introducción

El presente informe detalla el proceso de containerización, despliegue en un clúster de Kubernetes y auditoría de seguridad del sistema InvenTrack. Se documentan los pasos realizados para la creación de contenedores Docker para los componentes del sistema (backend, frontend y base de datos), la configuración de manifiestos de Kubernetes, el despliegue en un clúster local, y la configuración del Ingress para exponer la aplicación bajo el dominio `conjunta3p.espe.edu.ec`. Además, se presenta un análisis de seguridad siguiendo la metodología PTES (Penetration Testing Execution Standard), incluyendo la recopilación de inteligencia, modelado de amenazas, análisis de vulnerabilidades, explotación y post-explotación, así como las conclusiones y recomendaciones derivadas del proceso.

Objetivo General

El objetivo general de esta evaluación es realizar un despliegue seguro del sistema InvenTrack en un clúster de Kubernetes, asegurando la correcta containerización de sus componentes y evaluando su seguridad mediante pruebas de penetración siguiendo la metodología PTES.

Objetivos Específicos

- Realizar la containerización de los componentes del sistema InvenTrack (backend, frontend y base de datos) utilizando Docker.
- Configurar los manifiestos de Kubernetes necesarios para ejecutar InvenTrack en un clúster, incluyendo Deployments, Services, ConfigMaps, Secrets, PersistentVolumeClaims e Ingress.
- Configurar el Ingress para exponer la aplicación bajo el dominio `conjunta3p.espe.edu.ec` y asegurar la correcta resolución de DNS.
- Realizar una auditoría de seguridad del sistema InvenTrack siguiendo la metodología PTES, identificando vulnerabilidades y evaluando la efectividad de los controles de seguridad implementados.

Enlace al Repositorio de Manifiestos de Kubernetes

Repositorio GitHub (manifiestos .yaml):

https://github.com/IMarcusDev/Evaluacion_Final_k8s_30735_DSS/tree/main

Dockerización de Componentes

La containerización del sistema InvenTrack se organizó en tres componentes independientes: el backend (API REST en Node.js/Express), el frontend (aplicación React construida con Vite) y la base de datos MySQL. Para cada componente se definió una estrategia de empaquetado acorde con su función. El backend y el frontend se construyeron a partir de un **Dockerfile** propio, mientras que la base de datos se apoyó en la imagen oficial de MySQL 8.0, configurada por variables de entorno y volúmenes. En todos los casos se partió de imágenes base Alpine, que reducen el tamaño de la imagen resultante. A continuación se describen los Dockerfiles de cada componente y las decisiones de construcción aplicadas. Los términos técnicos de Docker y Kubernetes que aparecen en este informe se definen en el Glosario (Apéndice A), que puede consultarse para una mayor comprensión de los conceptos.

Backend (Node.js/Express)

```
1  #Marcos Escobar
2  FROM node:20-alpine
3
4  WORKDIR /app
5
6  COPY package*.json ./
7  RUN npm ci --omit=dev
8
9  COPY . .
10
11 RUN mkdir -p uploads \
12     && addgroup -g 1001 -S app \
13     && adduser -u 1001 -S app -G app \
14     && chown -R app:app /app
15 USER app
16
17 EXPOSE 4000
```

```
18  
19 CMD ["node", "src/app.js"]
```

Listing 1: Dockerfile del Backend

El Dockerfile del backend, presentado en el Listado 1, partió de la imagen `node:20-alpine`. Primero se copiaron los archivos `package*.json` y se ejecutó `npm ci --omit=dev`, de modo que la imagen incorpora solo las dependencias de producción (`--omit=dev` es la versión moderna de `--production`). Después se copió el código fuente y se creó el directorio `uploads`, destinado a las imágenes de producto que gestiona Multer. Un aspecto relevante para la seguridad fue la creación de un usuario sin privilegios (`app`, con UID y GID 1001) y la asignación de la propiedad del directorio de trabajo a dicho usuario. La instrucción `USER app` evitó que el proceso se ejecute como `root` dentro del contenedor, lo que reduce el impacto de una eventual intrusión. Por último se expuso el puerto 4000 y se definió el arranque del servidor con `node src/app.js`.

Frontend (React/Vite)

```
1 #Marcos Escobar  
2 FROM node:20-alpine AS build  
3 WORKDIR /app  
4 COPY package*.json ./  
5 RUN npm ci  
6 COPY . .  
7 RUN npm run build  
8  
9 FROM nginx:1.27-alpine  
10 COPY --from=build /app/dist /usr/share/nginx/html  
11 COPY nginx.conf /etc/nginx/conf.d/default.conf  
12 EXPOSE 80
```

Listing 2: Dockerfile del Frontend (multi-stage)

El Dockerfile del frontend, mostrado en el Listado 2, aplicó una construcción en dos partes (multi-stage). En la primera parte, basada en `node:20-alpine`, se instalaron las dependencias con `npm ci` y se generó la versión estática de producción con `npm run build`, que Vite genera en

el directorio `dist`. En la segunda etapa se utilizó la imagen `nginx:1.27-alpine` como servidor web. Solo los archivos compilados del directorio `dist` se copiaron a la ruta pública de Nginx, junto con una configuración personalizada (`nginx.conf`) que define el enrutamiento de la aplicación de página única y agrega cabeceras de seguridad. De este modo, la imagen final no contiene el código fuente ni las herramientas de compilación, lo que disminuye su tamaño y su superficie de exposición. El contenedor del frontend expone el puerto 80.

Base de Datos (MySQL)

La base de datos se ejecutó a partir de la imagen oficial `mysql:8.0`, sin un Dockerfile propio, dado que la configuración requerida se resolvió por variables de entorno y volúmenes. La persistencia de la información se garantizó con un volumen montado en `/var/lib/mysql`, de modo que los datos sobreviven al reinicio o a la recreación del contenedor. El esquema inicial se cargó de forma automática: los archivos de inicialización se montaron en el directorio `/docker-entrypoint-initdb.d`, que la imagen de MySQL ejecuta en la primera puesta en marcha cuando el directorio de datos está vacío. Ese conjunto incluyó el archivo `init.sql`, con la definición de las tablas, y un archivo de datos de ejemplo (`seed_01_electronica.sql`) que inserta usuarios, categorías, proveedores, productos y movimientos de kardex. Con este mecanismo, el sistema dispuso de datos iniciales sin una carga manual posterior. La orquestación de los tres contenedores se realizó con Docker Compose, que además definió un *healthcheck* sobre MySQL para que el backend inicie solo cuando la base de datos está operativa.

Evidencia: Contenedores en Ejecución

La construcción y el arranque de los tres contenedores se validaron con el comando `docker compose up --build`. La Figura 1 presenta la salida de `docker ps` tras el despliegue local. En ella se observa que los servicios `inventrack-frontend`, `inventrack-backend` e `inventrack-mysql` se encuentran en estado `Up`, con el contenedor de la base de datos marcado como `healthy`. Los puertos publicados confirman la exposición del frontend en el 5173, del backend en el 4000 y de MySQL en el 3306.


```

kernel      7.1.4-arch1-1
uptime      1 hour, 21 minutes
shell       fish
mem         11.44 GiB / 14.88 GiB
pkgs        1799
user        imarcus
hname       IMarcusDev
distro      Arch Linux

➜ 0s ○ docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"


| NAMES               | IMAGE                               | STATUS                 | PORTS                                                  |
|---------------------|-------------------------------------|------------------------|--------------------------------------------------------|
| inventrack-frontend | conjunta-desarrollo-seguro-frontend | Up 3 minutes           | 0.0.0.0:5173->80/tcp, [::]:5173->80/tcp                |
| inventrack-backend  | conjunta-desarrollo-seguro-backend  | Up 3 minutes           | 0.0.0.0:4000->4000/tcp, [::]:4000->4000/tcp            |
| inventrack-mysql    | mysql:8.0                           | Up 3 minutes (healthy) | 0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp |



➜ 0s ○ docker ps


| CONTAINER ID | IMAGE                               | NAMES               | COMMAND                  | CREATED        | STATUS                  | PORTS              |
|--------------|-------------------------------------|---------------------|--------------------------|----------------|-------------------------|--------------------|
| ef4afbee6455 | conjunta-desarrollo-seguro-frontend | inventrack-frontend | "/docker-entrypoint.s... | 54 minutes ago | Up 54 minutes           | 0.0.0.0:5173->80/t |
| 9a842d157be0 | conjunta-desarrollo-seguro-backend  | inventrack-backend  | "docker-entrypoint.s..." | 54 minutes ago | Up 54 minutes           | 0.0.0.0:4000->4000 |
| 904463ac13c3 | mysql:8.0                           | inventrack-mysql    | "docker-entrypoint.s..." | 54 minutes ago | Up 54 minutes (healthy) | 0.0.0.0:3306->3306 |


```

Figura 1

Contenedores en ejecución (docker ps).

Manifiestos de Kubernetes

El despliegue de InvenTrack en Kubernetes se definió mediante un conjunto de manifiestos en formato YAML. Los archivos se numeraron del 00 al 06 para fijar el orden de aplicación, de modo que los recursos base (namespace, secretos y configuraciones) existieran antes que los componentes que dependen de ellos. Cada archivo agrupó los recursos de una misma categoría: el namespace, los Secrets, los ConfigMaps, la base de datos MySQL con su volumen, el backend, el frontend y el Ingress. Esta separación permitió aplicar los manifiestos de forma ordenada con `kubectl apply`. A continuación se describe cada tipo de recurso y las decisiones de configuración aplicadas. El significado de cada objeto y parámetro de Kubernetes citado puede consultarse en el Glosario (Apéndice A).

Namespaces

Todos los recursos de InvenTrack se agruparon en un namespace propio llamado `conjunta-desarrollo-seguro`, cuya definición se presenta en el Listado 3. El namespace proporcionó un aislamiento dentro del clúster, de manera que los objetos del sistema quedaron separados de otras cargas de trabajo. Cada manifiesto posterior declaró de forma explícita este

namespace en su campo `metadata`, lo que evitó que los recursos se crearan en el espacio `default`.

Con ello se logró que la eliminación del namespace bastara para retirar por completo el despliegue.

```

1 apiVersion: v1
2 kind: Namespace
3 metadata:
4   name: conjunta-desarrollo-seguro

```

Listing 3: Definición del Namespace (00-namespace.yaml)

Deployments y Services

El sistema se estructuró en tres Deployments, uno por componente, cada uno acompañado de un Service de tipo `ClusterIP` que expuso el recurso solo dentro del clúster. La Tabla 1 resume las características principales de cada Deployment. El frontend se configuró con dos réplicas para tolerar la caída de un pod, mientras que el backend y la base de datos operaron con una sola réplica, dado que MySQL escribe sobre un volumen que no admite acceso concurrente desde varios pods.

Tabla 1

Resumen de los Deployments de InvenTrack

Deployment	Imagen	Réplicas	Puerto	Service
mysql	mysql:8.0	1	3306	ClusterIP
backend	inventrack-backend:1.0	1	4000	ClusterIP
frontend	inventrack-frontend:1.0	2	80	ClusterIP

El Deployment del backend, mostrado en el Listado 4, incorporó varias medidas de seguridad y de disponibilidad. Un `securityContext` obligó a que el contenedor se ejecutara como usuario sin privilegios (UID y GID 1001) y con `runAsNonRoot`, en concordancia con el usuario creado en el Dockerfile. Además, se deshabilitó el montaje automático del token de la cuenta de servicio (`automountServiceAccountToken: false`) para reducir la superficie de ataque en caso de intrusión. Un `initContainer` llamado `wait-for-mysql` retrasó el arranque del backend hasta que la base de datos respondiera, y las sondas `readinessProbe` y `livenessProbe` sobre la ruta

/api/health permitieron a Kubernetes verificar el estado del proceso. Las variables de entorno se inyectaron desde el ConfigMap y el Secret, sin valores en texto plano dentro del manifiesto.

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: backend
5    namespace: conjunta-desarrollo-seguro
6  spec:
7    replicas: 1
8    selector:
9      matchLabels: { app: backend }
10   template:
11     metadata:
12       labels: { app: backend }
13     spec:
14       automountServiceAccountToken: false
15       securityContext:
16         runAsNonRoot: true
17         runAsUser: 1001
18         runAsGroup: 1001
19         fsGroup: 1001
20       initContainers:
21         - name: wait-for-mysql
22           image: mysql:8.0
23           command:
24             - sh
25             - -c
26             - |
27               until mysqladmin ping -h mysql --silent; do
28                 echo "esperando a mysql..."; sleep 2;
29               done
30       containers:
31         - name: backend
32           image: inventrack-backend:1.0
33           ports:
```

```
34     - containerPort: 4000
35   env:
36     - name: DB_HOST
37       valueFrom: { configMapKeyRef: { name: inventrack-config, key:
38         DB_HOST } }
39     - name: DB_PASSWORD
40       valueFrom: { secretKeyRef: { name: inventrack-secrets, key:
41         DB_PASSWORD } }
42     - name: JWT_SECRET
43       valueFrom: { secretKeyRef: { name: inventrack-secrets, key:
44         JWT_SECRET } }
45   readinessProbe:
46     httpGet: { path: /api/health, port: 4000 }
47   livenessProbe:
48     httpGet: { path: /api/health, port: 4000 }
49   resources:
50     requests: { cpu: 100m, memory: 128Mi }
51     limits: { cpu: 500m, memory: 256Mi }
52 ---
53 apiVersion: v1
54 kind: Service
55 metadata:
56   name: backend
57   namespace: conjunta-desarrollo-seguro
58 spec:
59   type: ClusterIP
60   selector: { app: backend }
61   ports:
62     - port: 4000
63       targetPort: 4000
```

Listing 4: Deployment y Service del backend (04-backend.yaml)

Los Deployments de la base de datos y del frontend siguieron un patrón similar. En ambos casos se deshabilitó el token de la cuenta de servicio y se definieron límites de CPU y memoria, con el fin de evitar el consumo excesivo de recursos. El frontend, al servir contenido estático mediante

Nginx, requirió menos recursos y solo una sonda de disponibilidad sobre la ruta raíz. Cada Service asoció su selector de etiquetas con los pods correspondientes, lo que permitió que los componentes se comunicaran por nombre de servicio (`mysql`, `backend` y `frontend`) dentro del clúster.

ConfigMaps y Secrets

La configuración del sistema se separó en dos tipos de recursos según su confidencialidad. Los parámetros no confidenciales se almacenaron en el ConfigMap `inventrack-config`, presentado en el Listado 5, que definió el puerto del backend, el nombre del host de la base de datos, el usuario y el nombre de la base. Adicionalmente, un segundo ConfigMap (`mysql-initdb`) contuvo el archivo `init.sql` con el esquema y los datos iniciales, y se montó en el directorio de inicialización de la imagen de MySQL. De este modo, la información de configuración quedó desacoplada del código y de las imágenes.

```
1  apiVersion: v1
2  kind: ConfigMap
3  metadata:
4    name: inventrack-config
5    namespace: conjunta-desarrollo-seguro
6  data:
7    PORT: "4000"
8    DB_HOST: "mysql"
9    DB_USER: "root"
10   DB_NAME: "inventrack"
```

Listing 5: ConfigMap de configuración (02-configmap.yaml)

Los datos confidenciales se aislaron en el Secret `inventrack-secrets`, mostrado en el Listado 6, que agrupó la contraseña de la base de datos, la clave de firma de los tokens JWT y la clave de la API de Groq. El Secret se definió como `Opaque` y se empleó el campo `stringData` (véase el Apéndice A para la definición de ambos términos), que Kubernetes codifica en base64 al persistirlo. Conviene señalar que esta codificación no constituye un cifrado, por lo que los valores mostrados son de ejemplo y, en un entorno de producción, deberían gestionarse con un sistema de secretos dedicado. Tanto el ConfigMap como el Secret se consumieron mediante referencias `configMapKeyRef` y `secretKeyRef` en las variables de entorno de los contenedores, de manera que

ningún valor confidencial apareció en los manifiestos de los Deployments.

```
1  apiVersion: v1
2  kind: Secret
3  metadata:
4    name: inventrack-secrets
5    namespace: conjunta-desarrollo-seguro
6  type: Opaque
7  stringData:
8    DB_PASSWORD: "password"
9    JWT_SECRET: "jwt_secret"
10   GROQ_API_KEY: "groq_api_key"
```

Listing 6: Secret con credenciales (01-secrets.yaml)

PersistentVolumeClaim (PVC)

La persistencia de datos se resolvió con dos PersistentVolumeClaim, de modo que la información sobreviviera al reinicio o a la recreación de los pods. El primero, `mysql-pvc`, solicitó 2 Gi de almacenamiento y se montó en `/var/lib/mysql` para conservar la base de datos. El segundo, `uploads-pvc`, solicitó 1 Gi y se montó en `/app/uploads` del backend para almacenar las imágenes de producto que gestiona Multer. Ambos volúmenes utilizaron el modo de acceso `ReadWriteOnce`, tal como se muestra en el Listado 7. Por esta razón, el Deployment de MySQL adoptó la estrategia de actualización `Recreate`, que detiene el pod anterior antes de crear el nuevo y evita que dos pods intenten escribir sobre el mismo volumen de forma simultánea.

```
1  apiVersion: v1
2  kind: PersistentVolumeClaim
3  metadata:
4    name: mysql-pvc
5    namespace: conjunta-desarrollo-seguro
6  spec:
7    accessModes: ["ReadWriteOnce"]
8    resources:
9      requests:
10       storage: 2Gi
```

Listing 7: PersistentVolumeClaim de la base de datos (03-mysql.yaml)

Ingress

El acceso externo a InvenTrack se canalizó mediante un único recurso Ingress, denominado `inventrack-ingress`, cuya definición se presenta en el Listado 8. Este recurso empleó la clase `nginx` y publicó la aplicación bajo el dominio `conjunta3p.espe.edu.ec`. El enrutamiento se definió por rutas: las peticiones a `/api` y a `/uploads` se dirigieron al Service del backend en el puerto 4000, mientras que el resto de las rutas se enviaron al Service del frontend en el puerto 80. De esta forma, un solo punto de entrada atendió tanto la interfaz de usuario como la API. La anotación `proxy-body-size` se fijó en 10m para permitir la carga de imágenes de producto sin exceder el límite por defecto del controlador Ingress.

```
1 apiVersion: networking.k8s.io/v1
2 kind: Ingress
3 metadata:
4   name: inventrack-ingress
5   namespace: conjunta-desarrollo-seguro
6   annotations:
7     nginx.ingress.kubernetes.io/proxy-body-size: "10m"
8 spec:
9   ingressClassName: nginx
10  rules:
11    - host: conjunta3p.espe.edu.ec
12      http:
13        paths:
14          - path: /api
15            pathType: Prefix
16            backend:
17              service: { name: backend, port: { number: 4000 } }
18          - path: /uploads
19            pathType: Prefix
20            backend:
21              service: { name: backend, port: { number: 4000 } }
```

```
22     - path: /
23       pathType: Prefix
24       backend:
25         service: { name: frontend, port: { number: 80 } }
```

Listing 8: Definición del Ingress (06-ingress.yaml)

Despliegue en el Clúster y Configuración de Ingress

Descripción del Clúster

Controlador Ingress

Resolución del Dominio conjunta3p.espe.edu.ec

```
1 # >>> Ejemplo: 192.168.49.2    conjunta3p.espe.edu.ec <<<
```

Listing 9: Configuración de /etc/hosts para el entorno de prueba

Evidencia: Recursos en Kubernetes

Figura 2

Pods, Services e Ingress en ejecución (kubectl get all).

Evidencia: Aplicación en el Navegador

Figura 3

InvenTrack accesible bajo el dominio conjunta3p.espe.edu.ec.

Auditoría de Seguridad: Pentesting con Metodología PTES

Fase 1: Interacciones Preliminares (Pre-engagement) — 5 %

Alcance del Pentesting

Objetivos y Reglas de Juego

Activos a Probar

Autorización

Fase 2: Recopilación de Inteligencia (Intelligence Gathering) — 8 %

Mapeo de Puertos (Nmap)

```
1 # nmap -sV -p- conjunta3p.espe.edu.ec
2 # >>> PEGAR COMANDO Y RESULTADO <<<
```

Listing 10: Comando Nmap ejecutado

Figura 4

Resultado del escaneo de puertos con Nmap.

Detección de Tecnologías (WhatWeb / Wappalyzer)

Cabeceras HTTP y Rutas de Ingress

Fase 3: Modelado de Amenazas (Threat Modeling) — 7 %

Tabla 2

Matriz de modelado de amenazas de InvenTrack

Activo	Vector de Ataque	Superficie de Exposición
API REST		
Autenticación JWT		
Carga de archivos (Multer)		
Carga masiva Excel/CSV		
Chatbot (Groq)		
Base de datos MySQL		

Fase 4: Análisis de Vulnerabilidades (Vulnerability Analysis) — 12 %***Escaneo con Nikto***

```

1 # nikto -h https://conjunta3p.espe.edu.ec
2 # >>> PEGAR COMANDO Y RESULTADO <<<

```

Listing 11: Comando Nikto ejecutado

Escaneo con OWASP ZAP / Burp Suite***Análisis de Cabeceras de Seguridad******Resumen de Vulnerabilidades Detectadas*****Tabla 3**

Vulnerabilidades detectadas en la fase de análisis

ID	Descripción	Severidad (CVSS)
V-01		
V-02		
V-03		

Fase 5: Explotación (Exploitation) — 15 %***Intento de Bypass de Autenticación******Fuerza Bruta contra el Login (Hydra / Burp Intruder)***

```

1 # hydra -l admin -P rockyou.txt conjunta3p.espe.edu.ec http-post-form ...

```

Listing 12: Comando de fuerza bruta ejecutado

Inyección SQL (Login / Chatbot)

Pruebas de IDOR (Endpoints de Auditoría / Kardex)

Fase 6: Post-Explotación (Post-Exploitation) — 8 %

Análisis de Impacto

Validación del Hasheo con bcrypt

Escalación de Privilegios (admin vs almacenero)

Fase 7: Reporte (Reporting) — 5 %

Clasificación de Hallazgos por Severidad

Tabla 4

Hallazgos clasificados por severidad CVSS

ID	Hallazgo	CVSS	Severidad
H-01			
H-02			
H-03			

Validación de Controles de Seguridad

Tabla 5

Validación de controles de seguridad de InvenTrack

Control	Prueba Realizada	Resultado
Rate limiting		
Hasheo bcrypt		
Tokens JWT		
Verificación de sesión (5s)		

Conclusiones y Recomendaciones

Conclusiones

Recomendaciones de Mejora para InvenTrack

-
-
-

Apéndice A

Apéndice A: Glosario de Términos Técnicos

Este apéndice reúne la definición y la finalidad de los términos, comandos y objetos de Docker y Kubernetes que se mencionan a lo largo del informe. Su propósito es que el lector que no esté familiarizado con estas tecnologías pueda consultar aquí el significado de cada concepto antes de revisar las secciones técnicas. La Tabla A1 presenta los términos en orden alfabético, junto con una breve explicación de para qué sirven en el contexto del despliegue de InvenTrack.

Tabla A1

Glosario de términos técnicos empleados en el informe

Término	Definición y finalidad
<code>automount-</code>	Opción de un pod que controla si se monta de forma automática
<code>ServiceAccountToken</code>	el token de la cuenta de servicio de Kubernetes dentro del contenedor. Al fijarla en <code>false</code> se evita exponer ese token, lo que reduce la superficie de ataque si un contenedor es comprometido.
Alpine	Distribución de Linux minimalista que se usa como imagen base de contenedores. Su tamaño reducido disminuye el peso de la imagen final y su superficie de exposición.
Base64	Esquema que representa datos binarios como texto. Kubernetes almacena los Secrets en base64, pero conviene aclarar que se trata de una codificación reversible y no de un cifrado.
bcrypt	Función de derivación de hash diseñada para almacenar contraseñas de forma segura, con un factor de costo que dificulta los ataques de fuerza bruta.
ClusterIP	Tipo de Service que expone un conjunto de pods únicamente dentro del clúster, sin acceso directo desde el exterior.

Continúa en la página siguiente

Tabla A1 — continuación

Término	Definición y finalidad
ConfigMap	Objeto de Kubernetes que almacena parámetros de configuración no sensibles en pares clave-valor, desacoplados del código y de las imágenes.
Deployment	Objeto de Kubernetes que gestiona un conjunto de réplicas de pods y controla su creación, actualización y recuperación.
Docker	Plataforma de containerización que empaqueta una aplicación y sus dependencias en una imagen ejecutable de forma aislada.
Dockerfile	Archivo de texto con las instrucciones para construir una imagen de contenedor de manera reproducible.
Docker Compose	Herramienta que define y orquesta varios contenedores relacionados a partir de un único archivo de configuración.
Groq	Proveedor de inferencia de modelos de lenguaje que el asistente de InvenTrack utiliza para procesar consultas en lenguaje natural.
Healthcheck	Comprobación periódica que verifica si un contenedor funciona de forma correcta, empleada para condicionar el arranque de otros servicios.
Imagen	Plantilla inmutable a partir de la cual se crean los contenedores, que incluye el sistema de archivos y las dependencias de la aplicación.
Ingress	Objeto de Kubernetes que gestiona el acceso HTTP y HTTPS externo al clúster, con enrutamiento por host y por ruta hacia los Services internos.
<code>ingressClassName</code>	Campo del Ingress que indica qué controlador Ingress (por ejemplo, Nginx) debe atender las reglas definidas.

Continúa en la página siguiente

Tabla A1 — continuación

Término	Definición y finalidad
initContainer	Contenedor que se ejecuta y finaliza antes del contenedor principal de un pod, empleado para preparar condiciones previas como esperar a que la base de datos esté disponible.
JWT	Sigla de JSON Web Token. Token firmado que transporta la identidad del usuario y se usa para autenticar las peticiones a la API.
Kubernetes	Plataforma de orquestación que automatiza el despliegue, el escalado y la gestión de aplicaciones en contenedores.
livenessProbe	Sonda que Kubernetes utiliza para detectar si un contenedor sigue operativo. Si falla, el contenedor se reinicia.
Multer	Middleware de Node.js que procesa la carga de archivos, empleado en InvenTrack para las imágenes de producto.
Multi-stage build	Técnica de construcción de imágenes en varias etapas, que descarta las herramientas de compilación en la imagen final para reducir su tamaño.
Namespace	Partición lógica dentro de un clúster de Kubernetes que aísla y agrupa los recursos de una misma aplicación.
Nginx	Servidor web y proxy inverso, utilizado para servir el frontend estático y como controlador de Ingress.
<code>npm ci --omit=dev</code>	Comando que instala las dependencias exactas declaradas en el proyecto y omite las de desarrollo, de modo que la imagen incorpore solo lo necesario para producción.
Opaque	Tipo genérico de Secret en Kubernetes, destinado a almacenar datos sensibles arbitrarios como pares clave-valor. Es el tipo por defecto cuando la información no corresponde a un formato predefinido.

Continúa en la página siguiente

Tabla A1 — continuación

Término	Definición y finalidad
PersistentVolumeClaim (PVC)	Solicitud de almacenamiento persistente que un pod reclama al clúster, de forma que los datos sobreviven al reinicio o la recreación del pod.
Pod	Unidad mínima de despliegue en Kubernetes, que agrupa uno o varios contenedores que comparten red y almacenamiento.
proxy-body-size	Anotación del Ingress de Nginx que define el tamaño máximo del cuerpo de una petición, ajustada para permitir la carga de imágenes.
readinessProbe	Sonda que indica a Kubernetes cuándo un contenedor está listo para recibir tráfico.
ReadWriteOnce	Modo de acceso de un volumen que permite su montaje en lectura y escritura desde un solo nodo a la vez.
Recreate	Estrategia de actualización de un Deployment que elimina los pods anteriores antes de crear los nuevos, para evitar el acceso concurrente a un mismo volumen.
Réplica	Cada una de las copias de un pod que ejecuta un Deployment, empleadas para aumentar la disponibilidad y distribuir la carga.
runAsNonRoot	Ajuste del <code>securityContext</code> que obliga a ejecutar el contenedor con un usuario distinto de root, con lo que se limita el impacto de una intrusión.
securityContext	Sección del manifiesto que define los privilegios y permisos de seguridad con los que se ejecuta un pod o contenedor.
Secret	Objeto de Kubernetes que almacena datos sensibles, como contraseñas y claves, de forma separada de la configuración general.

Continúa en la página siguiente

Tabla A1 — continuación

Término	Definición y finalidad
Service	Objeto de Kubernetes que ofrece un punto de acceso estable, por nombre y dirección, a un conjunto de pods.
<code>stringData</code>	Campo del Secret que permite escribir los valores en texto claro dentro del manifiesto; Kubernetes los codifica en base64 al persistirlos.
Vite	Herramienta de construcción del frontend que genera la versión estática de producción de la aplicación React.
Volumen	Almacenamiento montado en uno o varios contenedores, que puede ser efímero o persistente según su tipo.

Apéndice B

Apéndice B: Evidencias Adicionales